

Data Preparation Session 4 Handout

Data structure – column/row operations

In our pipeline, we arrived to import the data, check and fix its general structure. We still work with the structure, but in small scale, changing one or a few columns at once.

If you want to keep or drop a few columns (variables), you can use `select()`.

- `select`: you can select **columns** (just like `SELECT` in SQL)
 - `:` for selecting a range of consecutive variables
 - `!` or `-` for taking the complement of a set of variables
 - `&` and `|` for selecting the intersection or the union of two sets of variables
 - `c()` for combining selections

If you want to use a subset of rows, your friend is `filter()`:

- `filter`: you can select **rows** (just like `WHERE` in SQL)
 - `==`, `>`, `>=` etc.
 - `&`, `|`, `!`, `xor()`
 - `is.na()` – see later
 - `between()`, `near()`

You can use the **`bind_rows()`** function from the `dplyr` package to bind together two data frames by their rows:

- `bind_rows(df1, df2, df3, ...)`
- `rbind` works well only with the same number of columns

Similarly, you can use the **`bind_cols()`** function from `dplyr` to bind together two data frames by their columns:

- `bind_cols(df1, df2, df3, ...)`
- `cbind` works well only with the same number of rows

You can also use `group_by()` and `summarise()` just like in SQL, and `arrange(dataframe, variable)`, just like `SORT BY` in SQL.

You may use `separate()` and `unite()` to separate one variable into many and unite many variables to a single variable.

`separate(data, col, into, sep, remove, convert)`

or with a more focused perspective

- `separate_wider_delim()` splits by delimiter:
 - `separate_wider_delim(data, cols, delim, names, cols_remove)`
- `separate_wider_position()` splits at fixed widths:
 - `separate_wider_position(data, cols, widths, cols_remove)`
- `separate_wider_regex()` splits with regular expression matches.
 - `separate_wider_regex(data, cols, patterns, cols_remove)`
 - As we will work with `Regex` in Session 8/9, you will be able to use its full capacities at the end of the course.

Let's start with the inpatient database.

First, we load it as in Session 2.

```
library(tidyverse)
names <- c("DRG", "ProviderID", "Name", "Address", "City", "State", "ZIP",
          "Region", "Discharges", "AverageCharges", "AverageTotalPayments",
          "AverageMedicarePayments")
types = 'ccccccciinnn'
inpatient <- read_tsv('inpatient.tsv',
                    col_names = names, skip=1, col_types = types)
```

We will work on the diagnosis-related group (DRG) variable. As the dataset has many rows, we can check the unique values.

```
unique(inpatient$DRG)
```

It seems that DRG column has two different things, a code and a description. Let's separate it. We will check different solutions to better understand how the separate() family works.

Let's try separating this on the hyphen

```
inpatient_separate <- separate(inpatient, DRG, c('DRGcode', 'DRGdescription'), '-')
```

We received warnings. Look at the error message and try to find out what is going on. We can investigate a problematic case. Let's look at row 45894.

```
inpatient$DRG[45894]
```

Yes, sometimes there are many hyphens. In this case, it's OK, as separate will take the first one, but it's a trap, the data is not always so easy.

Maybe adding the white space before and after the hyphen can solve the problem:

```
inpatient_separate <- separate(inpatient, DRG, c('DRGcode', 'DRGdescription'), ' - ')
```

Yes, we had good luck.

Let's separate with character position instead:

```
inpatient_separate <- separate(inpatient, DRG, c('DRGcode', 'DRGdescription'), 4)
```

Take a look at the data now

```
glimpse(inpatient_separate)
```

The DRGcode has an ending white space, while DRGdescription has a starting white space. We have to trim it (see in Session 8, text manipulation).

Alternatively, we can separate first the code, then the ' - '. This way, we will have three variables, we have to drop one.

```
inpatient_separate <- separate(inpatient, DRG, c('DRGcode', 'DRGdescription'), 3)
inpatient_separate <- separate(inpatient_separate, DRGdescription, c('waste', 'DRGdescription'), 3)
```

```
inpatient_separate <- select(inpatient_separate, -waste)
```

The same with the new function:

```
inpatient_separate <- separate_wider_delim(inpatient,
                                           cols=DRG,
                                           names=c('DRGcode', 'DRGdescription'),
                                           delim='-')
```

It does not work at all, we have to tell what to do with problematic values:

```
inpatient_separate <- separate_wider_delim(inpatient,
                                           cols=DRG,
                                           names=c('DRGcode','DRGdescription'),
                                           delim='-',
                                           too_many = "merge")
```

It makes the job.

Position works quite poor in this case::

```
inpatient_separate <- separate_wider_position(inpatient,
                                              cols=DRG,
                                              widths=c(DRGcode = 3, 3, DRGdescription=200),
                                              too_few="debug")
```

It gives the good results, but we have to remove the added columns:

```
inpatient_separate <- inpatient_separate %>% select(-c('DRG', 'DRG_ok', 'DRG_width',
'DRG_remainder'))
```

Let's see an example of the unite() function.

```
unite(data, col, ..., sep, remove)
```

where col stands for the new column to be created, and you list the columns to be united at ... You can also add a separator between the values, and you can remove or keep the original columns)

First, we load the inspections data (same as in Session 2):

```
names <- c("ID", "DBAName", "AKAName", "License", "FacilityType", "Risk", "Address",
           "City", "State", "ZIP", "InspectionDate", "InspectionType", "Results",
           "Violations", "Latitude", "Longitude", "Location")
```

```
inspections <- read_csv('inspections.csv',
                        col_names=names, skip=1)
```

And check its structure:

```
glimpse(inspections)
```

We would to create a new column called Regions that combines City and State (like Chicago, IL).

```
regional_inspections <- unite(inspections, Region, City, State, sep=" ", "
```

Let's look at the data

```
glimpse(regional_inspections)
```

If you don't want to DELETE the City and State columns:

```
regional_inspections <- unite(inspections, Region, City, State, sep=" ", remove=FALSE)
glimpse(regional_inspections)
```

Take a look at the unique regions:

```
unique(regional_inspections$Region)
```